Rho：从零设计一套研究工作台

给产品设计师的需求稿。重新设计应用结构、交互语言与插件体系，不以现有页面为实施目标。

一个完整的应用。
进入项目后，拥有自己的工作世界。

两层外壳

主应用与项目内部拥有清晰区分的导航和侧边栏。用户知道自己在哪个层级、哪个项目，以及如何返回。

三种工作模式

Studio 直接操作内容；Graph 理解全景关系；Agent 动态工作面组织任务、过程、结果与介入点。

一套插件语言

共享设计规范和交互习惯，保留每种插件的功能化特色。Agent 对话另有完整的专用组件设计。

> **委托范围**
>
> 上述范围来自本次用户澄清。后文将其展开为设计任务、场景和交付要求；具体布局、视觉语言、动效形式与组件细节由设计师探索。

当前前端与上一版 20 页稿均不作为参考布局或验收基线。保留的是不丢工作、状态真实、可追溯等产品行为底线。

用户要完成的工作，决定界面如何组织

首轮面向个人研究者与和 Agent 协作的分析者。多人实时协作不作为本次设计委托的隐含前提。

用户处境	需要完成的事	界面帮助其回答
跨项目工作	找到项目、恢复上次进度、知道哪些工作需要关注。	我在哪个项目？哪些状态属于哪个项目？
直接分析与制作	编辑代码、检查数据、调整图形、组织研究笔记。	我在改什么？哪些内容相关？如何并排检查？
理解研究过程	从一个结论追到数据、代码、运行和讨论。	它从哪里来？哪些是事实，哪些是建议？
委托与监督 Agent	表达目标、给上下文、跟进任务、必要时介入。	它在做什么？需要我做什么？产出了什么？
接续与判断	切换视角、离开再回来、理解结果并形成自己的判断。	刚才选择的对象、输入和工作位置还在吗？

内容模型

Project 容纳研究工作；数据、代码、运行、结果、问题和笔记是可关联的对象。文件是对象的来源之一，不是全部产品信息结构。

视图模型

同一个对象可以在 Studio、Graph 和 Agent 中呈现。视图切换改变看法，不创建另一份工作或另一套项目事实。

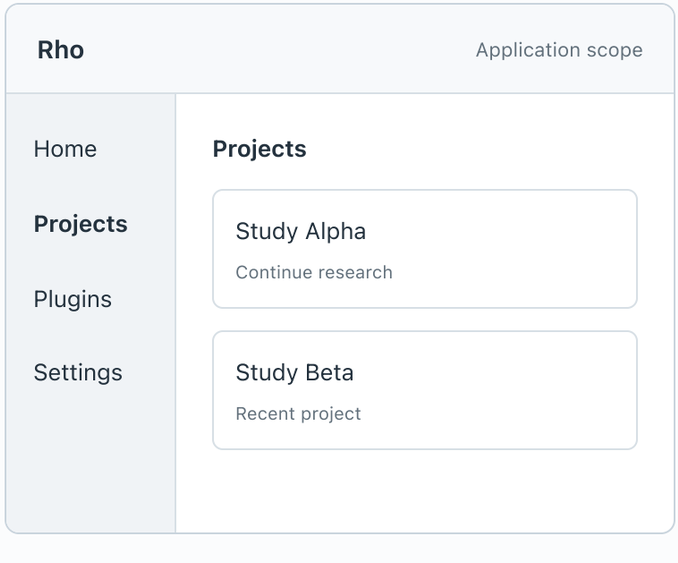
设计探索

设计师需要确认对象命名、信息层级和首屏主次。当前页面名称和文件目录树不构成最终信息架构。

先区分主应用，再进入一个项目

两个导航层级是明确需求。示意图展示作用范围；不规定只能使用双竖栏布局。

主应用 / 无活动项目

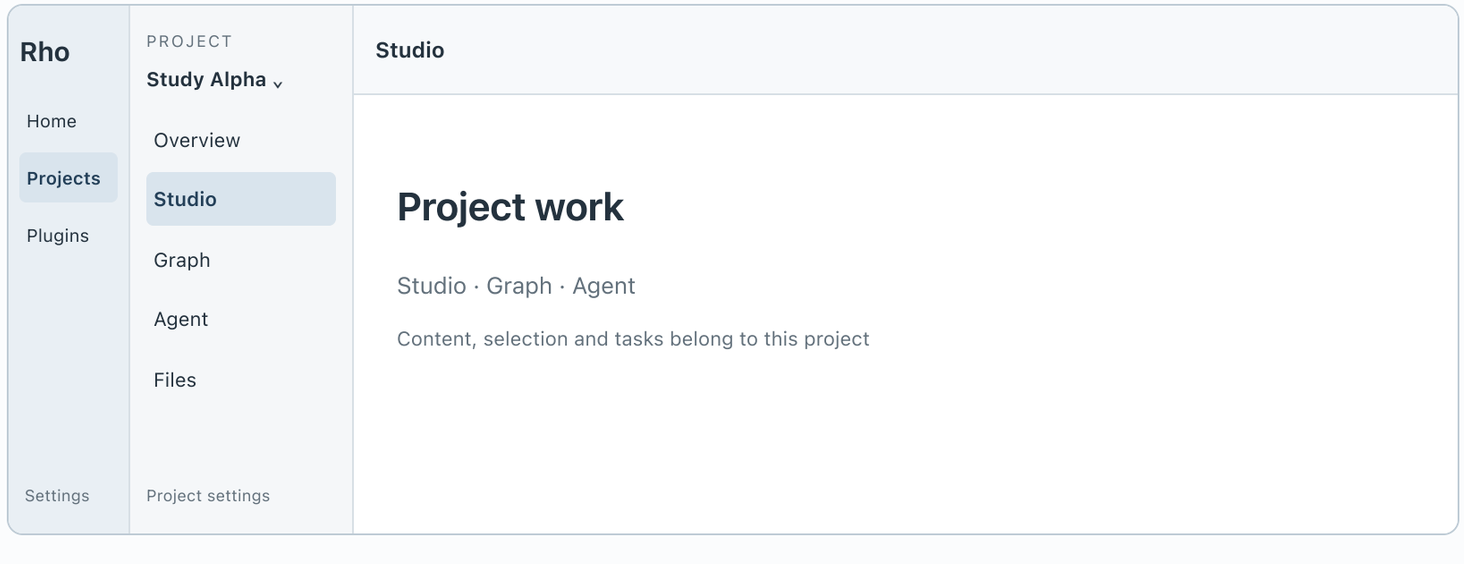


Enter project

→

Return to Projects

项目内 / Study Alpha



主应用侧栏

- 管理跨项目入口、最近工作与应用级能力。
- 项目卡与全局通知明确所属项目。
- 应用设置、插件管理与项目设置各有归属。

项目内侧栏

- 进入项目后有自己的导航、标题和工作入口。
- Studio、Graph、Agent 都属于当前项目。
- 项目切换、退出项目、模式切换含义不同。

需要设计师比较的方向

全局窄导航保留、项目导航并存；或进入项目后切换到项目外壳，保留明确返回入口。两种方案都必须让作用范围一眼可辨。

外壳需求：作用范围要在结构中成立

通过位置、选中态与切换行为体现层级。无需在每个按钮旁重复解释作用范围。

组件 / 范围	承担的任务	需要画出的关键转场
主应用导航	Home、Projects、应用级 Plugins 与 Settings；最终名称由设计师确定。	无项目 → 项目列表 → 进入项目；从项目返回。
项目切换与标题	始终能辨认当前项目；切换时知道旧项目有哪些工作仍在进行。	Alpha → Beta → Alpha；恢复原模式与工作位置。
项目内侧栏	概览、Studio、Graph、Agent、资源入口、项目设置。	首次进入、展开/收起、内容增加和空项目。
模式内导航	Studio 的场景/视图；Graph 的范围；Agent 的任务/会话。	切换局部对象；不复制出第三套全局菜单。
查找与定位	能找项目，也能找当前项目里的对象、任务与对话。	结果显示作用范围，打开后有清楚的返回路径。
上下文与状态	选中对象、详情、后台工作和需关注事件。	按需展开；不以常驻说明挤占内容。
插件入口	按用户任务找到功能，知道它作用于什么对象。	打开、加入工作面、不可用、移除后的既有内容。

设计边界

不把当前只有一个侧栏的实现当约束，也不预先锁定宽度、图标、颜色与菜单层数。用完整进入、返回和切换流程证明层级清楚。

Studio、Graph、Agent 是三种工作模式

同一项目、同一批对象，三个不同的主要任务，需要各自的界面组织。

MAKE & INSPECT

Studio

直接操作内容

Code | Data | Figure

UNDERSTAND & CONNECT

Graph

理解全景关系

Question → Data → Result

DELEGATE & INTERVENE

Agent

组织动态工作

Goal → Work → Result

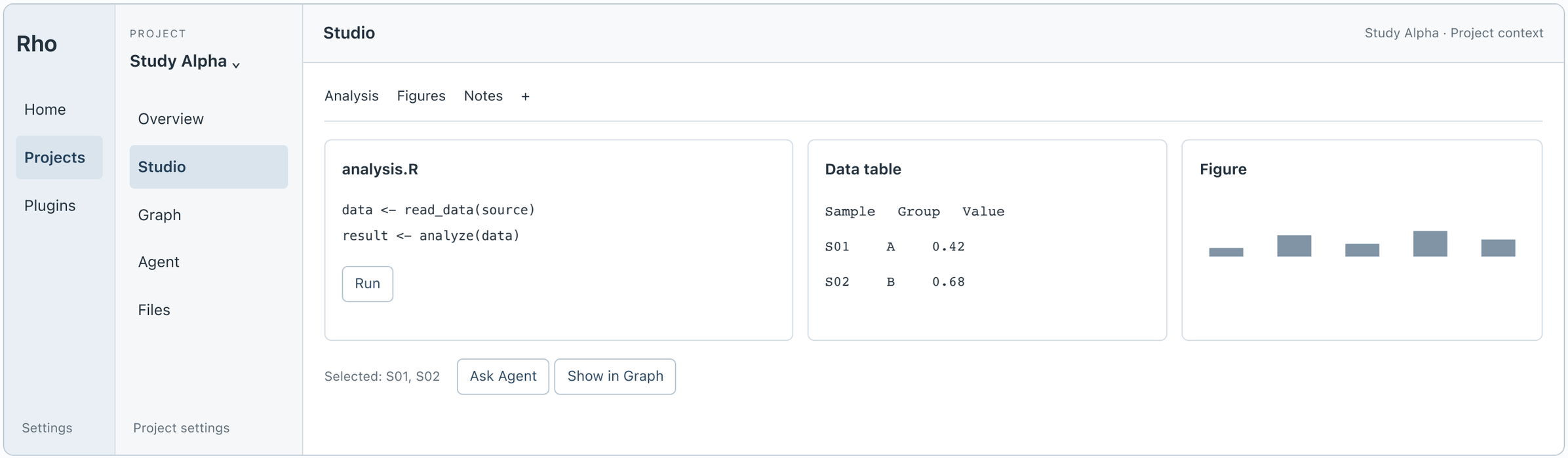
模式	主内容 / 主动作	跨模式连接
Studio	编辑器、表格、图形、笔记；编辑、选择、排列、比较。	选中内容带给 Agent；查看其在 Graph 中的关系。
Graph	对象、关系、分支、来源和路径；缩放、筛选、追溯、定位。	节点在 Studio 打开；路径或对象组带给 Agent。
Agent 动态工作面	目标、计划摘要、实际进展、结果与介入点；委托、查看、调整。	结果回到 Studio；有依据的关系进入 Graph。
Agent 专用对话组件	消息、上下文、输入、步骤与结果。它是交互组件，不是第四种模式。	在 Agent 中完整使用，在 Studio/Graph 中连续使用。

共同要求

切换模式保留项目、对象与上下文；分别恢复 Studio 布局、Graph 视角、Agent 当前任务和未发送输入。避免切换模式就像进入另一个产品。

Studio：一个可以真正工作的制作台

提供直接操作内容的稳定空间。研究对象和插件特性决定工作方式，不由聊天气泡统领所有操作。



需要设计

- 场景组织，视图打开、拆分、移动、关闭与布局恢复。
- 主焦点与选中对象清楚；其他视图可呈现相关内容。
- 专业工具、局部状态、保存与撤销有正确归属。
- 面板很窄、视图很多、长名称与大数据时如何退化。

需要回答

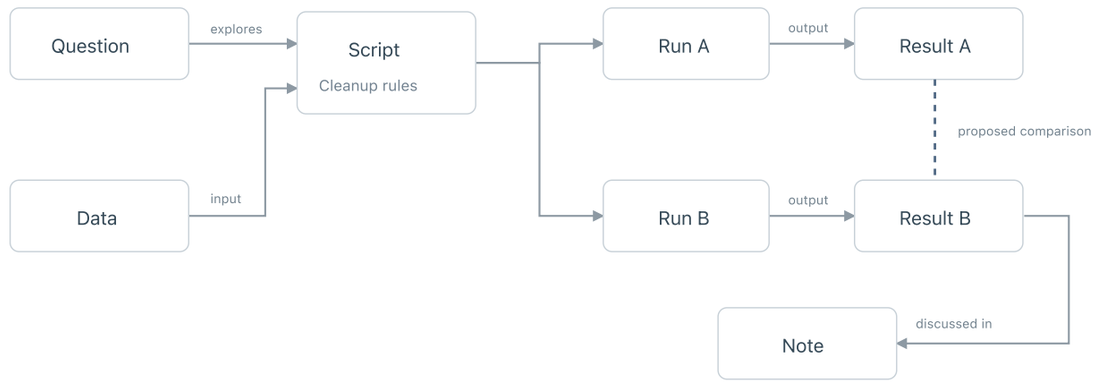
- 用户能否把代码、表和图作为同一次工作组织?
- Agent 更新是否打断正在输入的内容?
- 布局控制易发现，同时不挤占插件主内容吗?
- 打开同一对象两次，是否仍能理解它们的关系?

Graph：看见整个项目，而不是一团节点

全景用于定位、解释和发现关系。必须能从全局概貌缩小到一条可读的证据路径。

Project Graph

Overview　Focus path　Search



表达什么

- 数据、代码、运行、结果、问题、笔记等有意义的对象。
- 输入、产出、引用、派生、讨论等有类型的关系。
- 历史对象、当前对象与待验证建议有可辨认的状态。
- 已有关系与 Agent 建议明确分开；连线不能制造科学因果。

允许做什么

- 查找、按类型/任务缩小范围、折叠分组、聚焦上下游。
- 节点摘要与关系依据；打开 Studio 或带入 Agent。
- 从局部回全景，返回保留视角和选中对象。
- 大量节点仍可使用；提供可键盘操作的列表/大纲入口。

交付重点

展示全景、局部路径、节点详情、筛选、无关系、关系不完整和大项目场景。不能只交一张网络图。图中对象与关系是需求示例。

Agent 灵动界面：从轻量入口到动态工作面

界面随任务与状态有意义地变化。轻量入口与完整动态工作面如何结合，交由设计师探索。

01 / 准备

Ask Agent

目标与上下文

→

02 / 工作

Comparing rules

实际步骤与对象

→

03 / 介入

Input needed

问题与相关选择

→

04 / 结果

Results ready

答案与检查入口

轻量动态入口

人在 Studio 或 Graph 中工作时，用紧凑形态表达相关进展、结果和需介入事项；可展开查看，再收起回到原工作。

- 状态条、状态岛、局部卡片等是待比较的方向。
- 明确对应哪个任务；多任务不混成一个总状态。
- 可静默、收起、稍后查看；重要性与打断程度匹配。

完整动态工作面

在 Agent 模式中，围绕任务组织目标、计划摘要、实际动作、可操作结果与必要输入；按任务组合表、图、表单和说明。

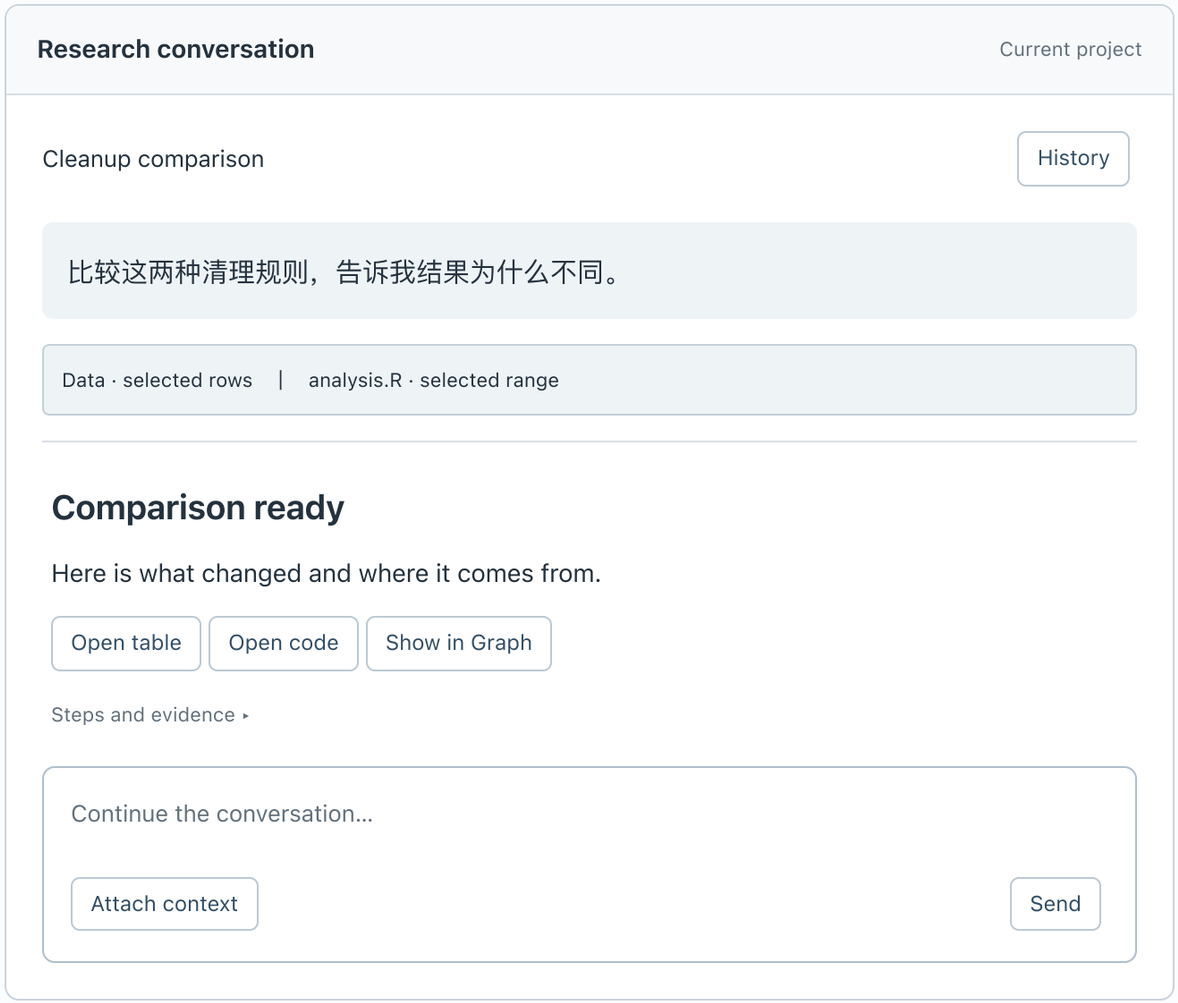
- 不把所有内容压成聊天气泡或日志列表。
- 结果直接进入专业插件，也能成为后续对话上下文。
- 新增内容不抢焦点；显示变化有来源与时间。

动效与真实状态

展开/收起和内容转换有连续性，并支持减少动态效果。只呈现实际状态与可解释的计划摘要，不制造“思考过程”、伪进度或装饰性的忙碌动画。

Agent 对话：一套专用的交互组件

单独设计结构、输入和结果语法。既能承载完整对话，也能与当前研究对象紧密配合。



组件必须覆盖

- 会话身份、标题、历史和当前任务的关系。
- 用户消息、Agent 回答、工具/步骤摘要、结构化结果与错误。
- 多行输入、附件、精确上下文、查看/移除、发送与未确认状态。
- 连续输出、停止请求、已确认停止和未知结果。
- 引用可打开；查看历史时不被新内容强制拉走。
- 中文 IME、键盘、选择复制、草稿恢复与焦点。
- 接入方式和输入通道如实表达，不画出不能送达的发送按钮。

需要交付的三种形态

完整对话工作面、与 Studio/Graph 并存的紧凑形态、从对象出发的上下文提问。三者共享同一会话与草稿语义，不各自发明一套聊天组件。

主应用组件：设计完整的产品骨架

这些是设计系统里的应用组件，服务多个工作模式；不等同于当前代码中的组件目录。

组件组	要解决的问题	设计稿应展示
应用导航与项目入口	跨项目定位、打开、最近访问、返回与应用管理。	首页/项目列表、进入与离开、空状态、搜索。
项目外壳与侧栏	项目身份、模式切换、资源入口、项目设置。	项目完整外壳；与全局导航的关系。
视图与布局容器	专业内容组合、焦点、拆分、切换、关闭与恢复。	单/多视图、窄面板、隐藏内容、键盘替代操作。
对象摘要与上下文	选了什么、准备交给谁、来自哪个版本。	单/多对象、范围、旧上下文、不可用对象。
结果与证据入口	结果可读、可操作、可追到生成过程。	表/图/代码/笔记、部分结果、比较与来源。
Agent 动态入口与任务容器	不离开工作也能跟进；需要时展开完整任务。	紧凑、展开、多任务、介入、完成、失败、断线。
Agent 专用对话	表达、上下文、消息与继续协作。	完整与紧凑形态、输入/流式/错误/历史/结果。
查找、菜单、详情与反馈	发现操作，理解状态与恢复途径。	搜索范围、菜单、对象详情、局部错误、通知。

> **与专业插件的分工**
>
> 外壳管理位置、范围、组合与跨模式协作；专业插件决定内容如何阅读和操作。插件不重复建设项目导航、会话系统与全局通知。

插件统一设计规范：一致性要落到交互

统一不只是一套颜色。定义可复用的视觉、行为和组合规则，并用不同插件检验。

需要统一的部分	需求	保留给插件的空间
身份与外框	对象标题、类型、状态、布局菜单与焦点有一致语法。	内容标题、对象摘要与专用信息。
动作层级	主动作、局部工具、更多操作与高影响操作有一致层级。	具体动作及其领域词汇。
选择与上下文	选中范围、附带给 Agent 的内容可见且可检查。	行、实体、代码范围、画布区域等选择方式。
结果与关系	可打开精确对象、查看来源、在 Graph 中定位。	领域结果格式、验证方法与解释。
状态与恢复	加载、空、失败、部分、过期、只读、不可用有可理解的语法。	真实领域条件与对应的修复动作。
输入与撤销	焦点、菜单、键盘、组合输入与撤销范围不相互抢占。	编辑器、网格、画布等成熟的专业习惯。
主题与密度	颜色语义、字级、间距、图标、对比度与焦点表达可复用。	密集数据、长文本、大画布的内容密度。
组合与尺寸	进入 Studio、从 Graph 打开、作为 Agent 结果时仍是同一种能力。	完整、紧凑、只读摘要等合理形态。

> **设计交付**
>
> 组件变体、设计 tokens、用法与禁用例，至少三种内容形态的组合示例。本稿不预先指定颜色、字体、圆角或技术框架。

每个插件回答同一张需求卡

可直接交给插件设计者的模板。规定必须说明的事情，不强迫所有插件使用同一种内部布局。

字段	设计者需要说明	Data table 示例
用户目标与对象	谁使用、解决什么问题、操作什么内容。	检查数据质量，比较两个结果。
专业特色	最值得突出的一到三个能力。	类型/缺失值、按键比较、快速筛选与选择。
入口与形态	从哪里进入；完整、紧凑、摘要分别做什么。	Studio 主视图；Graph 打开；Agent 结果摘要。
主内容与动作	首屏读什么、最重要的动作、进阶工具在哪里。	表与变化；筛选、比较、查看来源。
选择与协作	选择含义；交给 Agent、Graph 和其他插件什么。	实体键、列、范围与对象版本。
状态与连续性	显式状态；切换模式与返回保留什么。	筛选/排序/选择；部分数据和过期比较。
编辑与恢复	是否改变原内容；保存、撤销、冲突和失败。	浏览与修改区分；可编辑表设计保存语义。
尺寸与可访问性	窄面板、键盘、触控、长内容与大数据。	列多时可滚动；比较不依靠跨页记忆。
交接材料	核心操作、异常、跨模式串联与待建设能力。	选择 → 精确提问 → 比较 → Graph 来源。

> **实现方的责任**
>
> 设计需要当前系统尚不具备的能力时，记录为待建设能力并评估归属。不能把“现有接口没有”改写成产品不需要，也不能把设计图当成能力已实现。

插件实例：共同规范下的四种专业性

这些实例用于验证规范是否有用。名称描述功能，最终命名与内容分组可由设计师调整。

Code / 代码编辑

精确编辑 · 对比修改 · 执行联系

- 代码、范围选择、诊断和修改差异是主内容。
- 解释、重构、执行代码与查看输出有清楚关系。
- 共享编辑、自己的撤销、保存冲突与运行快照不混同。

需要画：人与 Agent 的修改对比；执行后定位相关结果。

Data / 数据表

结构与质量 · 实体选择 · 比较

- 列类型、缺失值、单位、过滤与排序帮助理解数据。
- 实体对齐比较，显示前后值、差值与未匹配项。
- 浏览状态与修改区分；大数据保持范围感。

需要画：选择一组实体提问；比较结果与窄面板。

Figure / 图形工作台

画布 · 映射与选择 · 图形输出

- 画布占主导；图例、映射与工具按任务出现。
- 数据图点选与原始图像区域选择语义不同。
- 数据选择可与表联动；静态图框选不推断数据实体。

需要画：点选回表；图像区域提问；导出与来源。

Notes / 研究笔记

连续写作 · 证据引用 · 研究判断

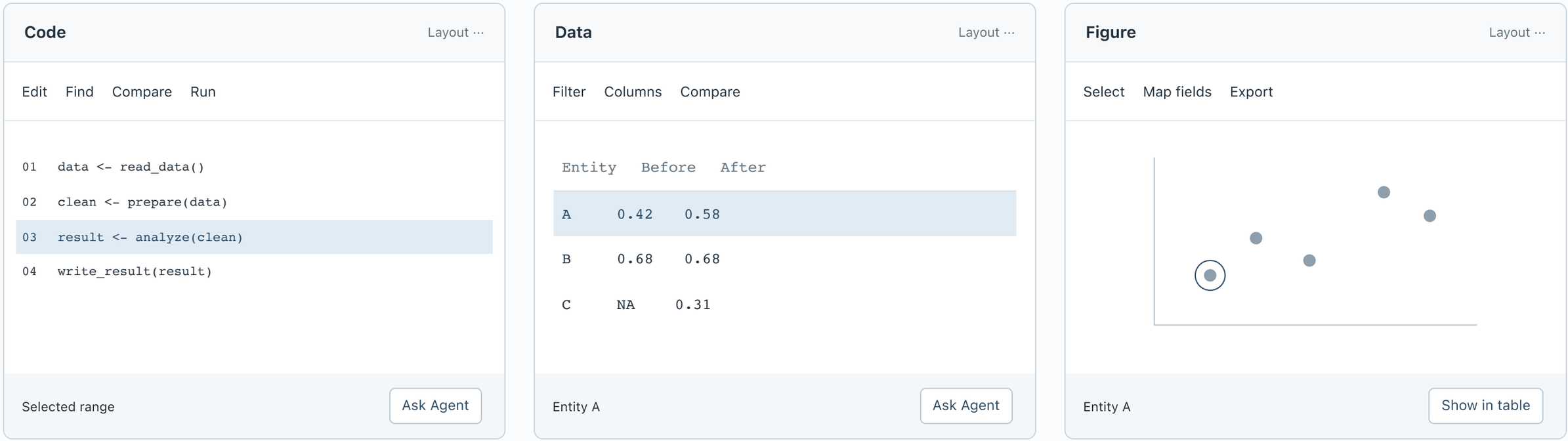
- 组织问题、方法、发现和待验证判断，支持长文。
- 引用表、图、运行与对话，回到具体对象。
- 人的文字和 Agent 建议可辨认，编辑/撤销/保存连贯。

需要画：引用结果进笔记，再从 Graph 找回讨论。

后续可用领域插件检验专业参数、质量检查与结果视图是否也能纳入规范；不把某个科学方法写死在主应用里。

同一外框，三种不同的内容组织

统一后仍要像专业工具；不同工具放进同一工作台，又能自然协作。



共享的部分

- 标题和布局入口；动作分组与焦点语法。
- 上下文和范围；结果、来源与 Graph 的连接。
- 加载/失败/部分结果、主题与可访问性。

应该不同的部分

- 编辑器保护输入；表格保护列与比较关系。
- 图形保护画布与选择；笔记保护阅读与写作。
- 每种插件采用其专业任务需要的局部控件。

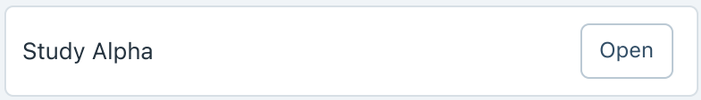
规范的边界

不能把所有插件包装成相同的表单、聊天卡片或通用信息面板。统一规范减少学习成本，保留专业功能。示意数据不代表研究结论。

分镜 A：进入项目，完成一次研究协作

场景：比较两种数据清理规则。从完整应用层级开始，而不是从一张已打开的表格开始。

01 / 主应用选择项目

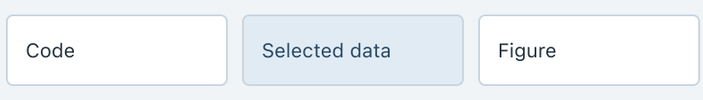


动作 从主应用进入 Study Alpha。

反馈 进入该项目外壳与独立侧栏。

连续性 全局返回入口仍清楚。

02 / Studio 直接检查



动作 打开数据与代码，选择记录。

反馈 专业视图并存，范围可辨认。

连续性 布局与未保存内容保留。

03 / 专用对话表达目标

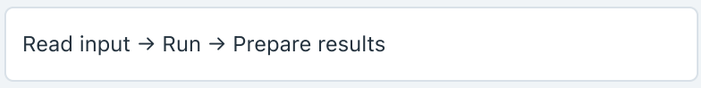


动作 检查附件，向选定 Agent 发任务。

反馈 目标、对象与消息连接到任务。

连续性 输入与发送状态可恢复。

04 / 动态工作面跟进

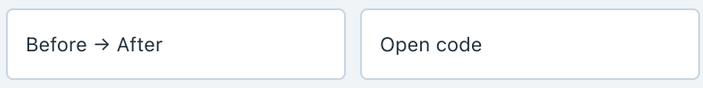


动作 需要时展开 Agent，其余继续工作。

反馈 呈现进展和相关结果。

连续性 更新不抢占 Studio 焦点。

05 / Studio 验证结果

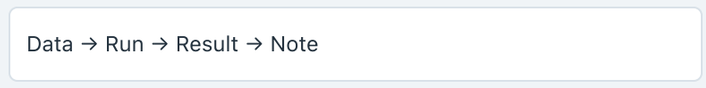


动作 检查变化，核对代码与图。

反馈 比较对齐同一对象，来源可到达。

连续性 原结果仍可回看。

06 / Graph 与笔记收束



动作 追溯工作，把判断写进笔记。

反馈 关系说明来源，笔记承载人的结论。

连续性 返回 Studio 保留工作位置。

分镜 B：从全景关系进入一个具体问题

场景：几周后回到项目，研究者想知道一个图形为何产生，哪些工作受输入变化影响。

01 / 全景定位

Project Graph

Questions　Data　Runs　Results　Notes

动作　查找图形，缩小到相关分支。

反馈　全局概貌变为有类型、可读的局部关系。

连续性　保留返回全景的位置。

02 / 沿证据路径追溯

Figure → Result → Run → Script → Data

Open relation evidence

动作　查看输入、脚本与运行的关联依据。

反馈　事实与建议、当前与历史对象可区分。

连续性　不把连线当成科学因果。

03 / 带路径给 Agent

如果替换这份输入，哪些结果需要重新检查?

Context: selected path

动作　将准确路径和对象带入对话。

反馈　动态工作面呈现分析与建议范围。

连续性　图的视角和对话上下文可返回。

04 / 在 Studio 检查建议

Original figure　Candidate result

Return to Graph

动作　检查候选结果，再记录判断。

反馈　结果与建议有来源；Graph 不伪造事实。

连续性　旧结果、路径和笔记可访问。

要讲清楚的动作链

从哪里进入 Graph、怎样聚焦、看什么详情、如何带走上下文、做完怎样返回。它是可行动的项目视角，不能只是 Studio 的背景皮肤。

分镜 C：Agent 工作，用户继续自己的事

场景：长任务在进行中，用户切换对象或项目。灵动入口提供适度的连续感与介入机会。

01 / 收起后继续工作

Studio / editing notes

Agent · Comparing rules · Running

动作 收起完整工作面，继续写作。

反馈 紧凑入口只显示必要状态，可按需展开。

连续性 笔记、光标与草稿不改变。

02 / 需要人介入

Input needed · Choose scope

Review

动作 打开与任务相关的问题。

反馈 给出已有上下文与明确选择。

连续性 已有授权不因界面切换再请求一遍。

03 / 连接或确认丢失

Connection lost

Showing received work

Reconnect

动作 断线后读已收到的内容，再重连。

反馈 未知、失败、停止请求和已停止分开。

连续性 观察原任务，不重复发起工作。

04 / 结果与返回

Results ready

Open result | Later | Conversation

动作 现在检查结果，或稍后再处理。

反馈 通知连接同一任务、对话与相关对象。

连续性 回到原编辑位置继续工作。

多任务与跨项目

紧凑入口标明对象范围；跨项目通知先说明项目，再进入对应上下文。停止只针对明确选中的任务；关闭入口、关闭视图和停止执行不混同。

异常、输入和尺寸变化，属于同一套体验

在设计稿和可点击原型中展示，不留到开发时临时补一句错误提示。

场景	设计需要表达	可继续的动作
首次使用 / 空项目	主应用与项目层级仍然成立，知道从何开始。	打开/创建项目，添加内容，进入 Studio 或发起任务。
Graph 很空或很大	无关系、关系尚不完整、只显示局部是不同情况。	查找、筛选、聚焦、回全景，使用替代列表。
Agent 等待 / 无回答	未发送、发送未确认、等待、无可靠终态分别表达。	保留输入与已收到内容，检查原任务。
冲突 / 部分结果 / 不可用	影响范围与已保留内容清楚，不封锁整个项目。	比较修改、看已有输出、回来源、选可用入口。
空间变窄 / 内容变长	导航层级、主对象与返回路径仍可辨认。	收起次要面板，专注视图；前后值不分开记忆。
键盘 / 中文输入 / 减少动态效果	焦点明确，输入组合完整，状态不只靠颜色或动画。	菜单替代拖动、图对象列表、可控通知与动效。

尺寸交付

桌面研究工作为主要对象，同时覆盖紧凑窗口与单个窄插件面板。手机阅读和轻量反馈采用简化方案，不机械缩小桌面截图。

文案原则

产品界面使用 English；研究内容保留原始 Unicode。让结构解释作用范围，文案用于具体动作、必要状态和恢复，不堆重复副标题。

设计师的工作包：先探索，再形成系统

委托的是产品设计。设计师对信息架构和交互负责，不是照着当前截图换皮。

阶段	必须交付	评审关注
A / 结构探索	用户任务、产品对象、两层架构；至少两种有实质差别的外壳和灵动界面方向。	层级和模式是否清楚，方案取舍理由是否成立。
B / 核心流程	进入项目；Studio、Graph、Agent 各自的关键帧；三条端到端可点击流程。	能否带着上下文往返，失败后能否继续。
C / 组件与插件系统	主应用组件、专用对话、动态 Agent、插件通用规范和四个功能化实例。	统一是否减少学习成本，专业能力是否保留。
D / 细化交接	选定方向的视觉稿、组件变体、tokens、动效/减少动态效果、尺寸、状态与行为标注。	开发是否知道动作效果、状态来源和需保留的内容。
E / 验证与修订	观察未参与设计的人做代表性任务；记录卡点，交付修订、决策和待建设能力清单。	以人的理解和完成情况反馈，不以像不像当前界面判断。

本轮核心范围

主应用与项目侧栏、三种工作模式、灵动 Agent、专用对话、插件规范、实例插件与跨模式分镜必须齐全。

交给设计师决策

导航布局、Graph 的组织和缩放方式、灵动入口形态、对话嵌入方式、视觉语言与命名。用方案和场景解释选择。

评审需求与设计成果，看这些问题

目标来自未来产品需要。实施团队再为选定体验建设能力；当前实现不决定应当设计什么。

必须能够清楚演示

- 区分主应用和项目内部；进入、返回、切换项目清楚。
- Studio、Graph、Agent 各有主要任务与界面组织。
- Graph 能追溯和行动；Agent 随状态变化并允许恰当介入。
- 对话是完整组件，输入、上下文、步骤与结果连续。
- 插件既有统一规范，又保留代码、表、图、笔记的专业差异。
- 跨模式、离开返回、异常恢复，保留人的输入和已获得工作。

产品行为底线

- 对象有各自身份、版本与来源；没有代表全项目科学正确的“版本”。
- 同一文档多视图共享内容，保存、撤销与冲突可理解。
- 任务、消息、重连保持原身份；未确认结果不伪装成功或自动重做。
- 已有授权按实际范围执行；模型和界面建议不能扩张权限。
- 执行结束、产物完整、领域检查与 Agent 解释分别有依据。
- 动态界面组织真实能力与内容，不把视觉效果当执行或授权。

> **文档地位与修订**
>
> v2 按用户澄清重写为从零设计需求稿，取代上一版基于现状的设计提案。功能展开用于设计委托讨论；示意框图不是视觉定稿，也不要求沿用任何当前 SDK、菜单或布局。

设计师交出体验与设计系统。实施方识别能力缺口、实现并验证。共同对最终产品负责。